

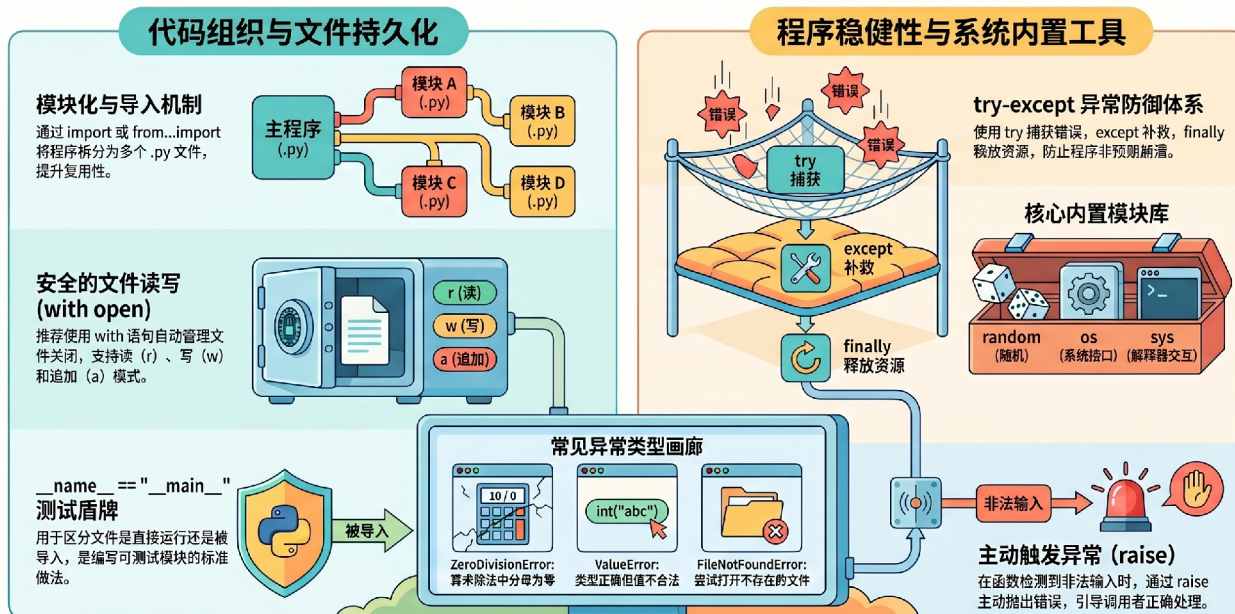
模块、内置模块、文件操作以及异常处理



学习目标：

1. 理解“模块”的概念，学会创建和使用模块。
2. 掌握 `import`、`from...import`、`as` 导入模块的方式，理解如何访问模块成员。
3. 理解 `if __name__ == "__main__"` 的作用。
4. 了解包（Package）的基本结构。
5. 掌握 Python 常用内置模块：`random`、`math`、`datetime`、`os`、`sys`。
6. 掌握文件读写操作：`open`、读/写/追加模式、`with` 语句。
7. 理解异常的概念，能够使用 `try-except` 捕获并处理异常。
8. 能结合模块、文件操作和异常处理编写实用程序。

Python 编程进阶：模块化、文件操作与异常处理全书



第九章：模块与包

1. 为什么需要模块？

定义：模块（Module）是一个包含 Python 代码的 `.py` 文件。模块化编程就是把一个大的程序拆分成多个小的文件，每个文件负责一部分功能。

生活类比：写一本 1000 页的书，如果全部写在一个文件里，翻看、修改都极其困难。更好的做法是分成“第1章”、“第2章”……每个章节单独一个文件。模块就相当于书中的“章”。

好处：

- 代码结构清晰，易于维护。
- 可以重复使用：写好的模块可以被多个项目导入。
- 避免命名冲突：不同模块可以有同名函数。

2. 什么是模块？

定义：一个 `.py` 文件就是一个模块。模块名就是文件名（不带 `.py`）。

示例：创建一个 `my_math.py` 文件：

```
python
1  # my_math.py
2  def add(a, b):
3      return a + b
4
5  def mul(a, b):
6      return a * b
7
8  PI = 3.14159
```

这个文件就是一个名为 `my_math` 的模块。

3. 如何导入模块并访问其中的成员？

3.1 使用 `import` 模块名

导入整个模块，使用时需要加上 `模块名.` 前缀来访问模块中的函数、变量。

```
python
1 import my_math
2
3 # 访问模块中的函数和变量
4 print(my_math.add(5, 3))    # 8
5 print(my_math.PI)          # 3.14159
```

解读：`my_math.add` 表示“在 `my_math` 模块里的 `add` 函数”。**必须通过模块名加句号**才能使用模块内部的内容。

3.2 使用 `from 模块名 import 成员名`

只导入模块中的部分成员（函数、变量等），使用时**不需要**模块名前缀，直接写成员名即可。

```
python
1 from my_math import add, PI
2
3 print(add(5, 3))    # 8, 不需要写 my_math.add
4 print(PI)          # 3.14159
5 # print(mul(2,3))  # 报错, 因为 mul 没有被导入
```

④ Note

这种方式更简洁，适合只用到少数几个成员的情况。

3.3 使用 `as` 给模块或成员起别名

当模块名太长或想避免命名冲突时，使用别名。

```
python
```

```

1  import my_math as mm
2  print(mm.add(10, 20))    # 30
3
4  from my_math import add as my_add
5  print(my_add(1, 2))      # 3

```

3.4 不推荐：from 模块名 import *

这会导入模块中的所有成员，容易导致命名冲突，且代码可读性差。

```

python
1  from my_math import *
2  print(add(2,3))          # 能用，但不知道 add 来自哪里

```

Important

访问成员的总结：

- `import 模块名` → 通过 `模块名.成员` 访问。
- `from 模块名 import 成员` → 直接使用成员名，无需前缀。
- 无论哪种方式，只有被导入的成员才能被访问；未导入的成员无法使用。

4. if __name__ == "__main__" 的作用

定义： `__name__` 是 Python 内置变量，表示当前模块的名字。当直接运行一个 `.py` 文件时，`__name__` 的值被设为 `"__main__"`；当该文件被作为模块导入时，`__name__` 的值就是模块名（即文件名）。

大白话： 你可以写一段测试代码，只有当你直接运行这个文件时才执行；别人导入你的模块时，测试代码不会自动运行。

示例： 修改 `my_math.py`：

```

python
1  # my_math.py
2  def add(a, b):
3      return a + b

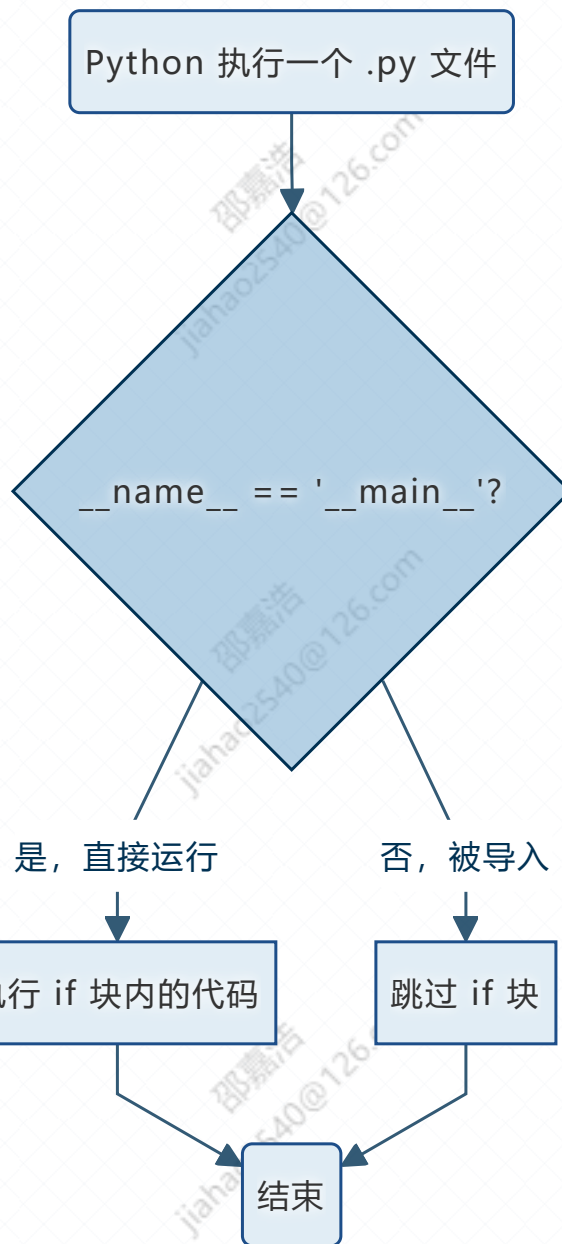
```



```
4
5  def mul(a, b):
6      return a * b
7
8  PI = 3.14159
9
10 if __name__ == "__main__":
11     # 以下代码只在直接运行 my_math.py 时执行
12     print("测试: ", add(10, 20))
13     print("测试: ", mul(2, 3))
```

- 当你在命令行运行 `python my_math.py` 时，会输出测试信息。
- 当其他文件 `import my_math` 时，测试代码不会运行。

流程图：



💡 Tip

养成在每个模块末尾加上 `if __name__ == "__main__":` 的习惯，方便测试和调试。

5. 包 (Package) —— 多个模块的集合

定义：包是一个目录（文件夹），里面包含多个 `.py` 模块文件，并且必须有一个 `__init__.py` 文件（可以为空）。包用于将功能相近的模块组织在一起。

生活类比：书不仅有“章”，还可以有“篇”（篇下面有章）。包就像“篇”，模块就像“章”。

目录结构示例：

```
1 my_package/          # 包名
2     __init__.py      # 表示这个目录是一个包（可以是空文件）
3     math_utils.py    # 模块
4     string_utils.py  # 模块
```

导入包中的模块：

```
python
1 # 方式1：导入整个模块（需要包名, 模块名）
2 import my_package.math_utils
3 my_package.math_utils.add(1, 2)
4
5 # 方式2：导入模块并起别名
6 import my_package.math_utils as mu
7 mu.add(1, 2)
8
9 # 方式3：从包中直接导入模块
10 from my_package import math_utils
11 math_utils.add(1, 2)
12
13 # 方式4：从模块中导入具体函数
14 from my_package.math_utils import add
15 add(1, 2)
```

④ Note

Python 3.3+ 中，`__init__.py` 不再是必须的，但为了兼容性和清晰性，建议保留。

6. 课堂练习（模块部分）

1. **创建模块**：新建一个文件 `str_tools.py`，在里面定义两个函数：
- `reverse_string(s)`：返回字符串 `s` 的反转。
 - `count_vowels(s)`：返回字符串中元音字母（a,e,i,o,u）的个数（不区分大小写）。
- 然后在另一个文件 `test_str.py` 中导入该模块，测试这两个函数。
2. **使用** `if __name__ == "__main__"`：修改 `str_tools.py`，添加测试代码，使得直接运行该文件时输出测试结果，而被导入时不输出。

第 十 章：常用内置模块

Python 自带了很多有用的模块，直接 `import` 即可使用。以下介绍最常用的几个。

1. random 模块 —— 随机数

定义：`random` 模块提供了生成随机数的函数。

函数	说明	示例
<code>random.random()</code>	返回 0.0 到 1.0 之间的随机浮点数	<code>print(random.random())</code>
<code>random.randint(a, b)</code>	返回 a 到 b 之间的随机整数（包含两端）	<code>random.randint(1, 100)</code>
<code>random.choice(seq)</code>	从序列中随机选择一个元素	<code>random.choice(["红", "绿", "蓝"])</code>
<code>random.shuffle(lst)</code>	随机打乱列表的顺序	<code>random.shuffle(cards)</code>

python

```
1 import random
2
3 print(random.randint(1, 10))           # 随机整数 1~10
```



```
4 print(random.choice(["苹果", "香蕉", "橙子"])) # 随机水果
```

2. math 模块 —— 数学运算

定义：`math` 模块提供了常用的数学函数和常数。

函数/常数	说明	示例
<code>math.sqrt(x)</code>	平方根	<code>math.sqrt(25)</code> → 5.0
<code>math.pow(x, y)</code>	x 的 y 次方（也可用 <code>x**y</code> ）	<code>math.pow(2,3)</code> → 8.0
<code>math.pi</code>	圆周率 π	<code>print(math.pi)</code>
<code>math.e</code>	自然常数 e	<code>print(math.e)</code>
<code>math.sin(x)</code>	正弦（x 为弧度）	<code>math.sin(math.pi/2)</code> → 1.0

```
python
1 import math
2
3 print(math.sqrt(16)) # 4.0
4 print(math.pi)      # 3.141592653589793
```

3. datetime 模块 —— 日期和时间

定义：`datetime` 模块提供了处理日期和时间的类。

```
python
1 from datetime import datetime, date, time, timedelta
2
3 # 获取当前日期和时间
4 now = datetime.now()
5 print(now) # 2025-01-15 14:30:00.123456
6 print(now.year, now.month, now.day) # 年、月、日
7 print(now.hour, now.minute) # 时、分
```

```

8
9 # 创建指定日期
10 d = date(2025, 5, 1)
11 print(d) # 2025-05-01
12
13 # 日期加减
14 tomorrow = now + timedelta(days=1)
15 print(tomorrow)
16
17 # 格式化字符串
18 print(now.strftime("%Y-%m-%d %H:%M:%S")) # 2025-01-15
14:30:00

```

4. os 模块 —— 操作系统接口

定义： `os` 模块提供了与操作系统交互的功能（文件和目录操作、环境变量等）。

函数	说明	示例
<code>os.getcwd()</code>	获取当前工作目录	<code>print(os.getcwd())</code>
<code>os.listdir(path)</code>	列出目录下的文件和文件夹	<code>os.listdir(".")</code>
<code>os.mkdir(path)</code>	创建目录	<code>os.mkdir("test_dir")</code>
<code>os.remove(path)</code>	删除文件	<code>os.remove("data.txt")</code>
<code>os.path.exists(path)</code>	判断文件或目录是否存在	<code>os.path.exists("myfile.py")</code>
<code>os.path.join(a, b)</code>	拼接路径（跨平台安全）	<code>os.path.join("folder", "file.txt")</code>



python

```

1 import os
2
3 # 获取当前目录

```

```

4     current = os.getcwd()
5     print("当前目录:", current)
6
7     # 列出当前目录下的所有文件
8     files = os.listdir(".")
9     print("文件列表:", files)
10
11    # 判断文件是否存在
12    if os.path.exists("test.txt"):
13        print("文件存在")
14    else:
15        print("文件不存在")

```

5. sys 模块 —— 系统相关

定义：`sys` 模块提供了与 Python 解释器相关的变量和函数。

变量/函数	说明	示例
<code>sys.argv</code>	命令行参数列表	<code>python script.py a b</code> → <code>["script.py", "a", "b"]</code>
<code>sys.exit()</code>	退出程序	<code>sys.exit(0)</code> 正常退出
<code>sys.path</code>	模块搜索路径列表	<code>print(sys.path)</code>
<code>sys.version</code>	Python 版本信息	<code>print(sys.version)</code>

```

1     import sys
2
3     print("Python 版本:", sys.version)
4     print("命令行参数:", sys.argv)
5
6     # 强制退出（通常用于错误时）

```

python

```
7 | if len(sys.argv) < 2:
8 |     print("请提供参数")
9 |     sys.exit(1)
```

6. 课堂练习（内置模块）

1. **随机数模拟**：使用 `random` 模块生成 10 个 1~100 之间的随机整数，输出其中最大值、最小值和平均值。
2. **日期计算器**：输入你的出生日期（如 `2000-05-20`），计算到今天为止活了多少天。
3. **目录创建**：使用 `os` 模块创建一个新文件夹 `my_data`。
4. **命令行参数**：编写一个程序，接收两个命令行参数（两个数字），输出它们的和。如果参数不足，用 `sys.exit()` 提示。

第十一章：文件操作

1. 为什么需要文件操作？

定义：程序运行时产生的数据默认保存在内存中，程序结束就会丢失。文件操作可以将数据永久保存到磁盘（硬盘）上，下次程序运行时可以读取。

生活类比：就像用笔记本记笔记。内存相当于大脑的临时记忆（关机就忘），文件相当于写在纸上的文字（可以长期保存）。

2. 打开文件：`open()` 函数

语法：

```
python
1 | 文件对象 = open(文件名, 模式, encoding="编码")
```

常用模式：

模式	说明	文件不存在时的行为
'r'	只读（默认）	报错 <code>FileNotFoundError</code>
'w'	写入（覆盖原内容，若文件存在则清空）	创建新文件
'a'	追加（在末尾添加）	创建新文件
'x'	独占创建（文件已存在时报错）	创建新文件
'b'	二进制模式（如 <code>'rb'</code> 、 <code>'wb'</code> ）	-
't'	文本模式（默认）	-

编码：一般使用 `encoding="utf-8"` 支持中文。

python

```
1 # 打开文件（只读模式）
2 f = open("data.txt", "r", encoding="utf-8")
3 # 使用完毕后必须关闭
4 f.close()
```

Important

打开文件后一定要记得关闭，否则可能造成资源泄露或数据丢失。

3. 读取文件

3.1 `read()` —— 读取全部内容

python

```
1 f = open("data.txt", "r", encoding="utf-8")
2 content = f.read() # 读取整个文件内容为一个字符串
3 print(content)
4 f.close()
```

3.2 readline() —— 一次读一行

```
python
1 f = open("data.txt", "r", encoding="utf-8")
2 line = f.readline() # 读取第一行
3 while line:
4     print(line, end="")
5     line = f.readline()
6 f.close()
```

3.3 readlines() —— 读取所有行，返回列表

```
python
1 f = open("data.txt", "r", encoding="utf-8")
2 lines = f.readlines() # 列表，每个元素是一行（包含换行符）
3 for line in lines:
4     print(line, end="")
5 f.close()
```

3.4 使用 with 语句（推荐，自动关闭文件）

```
python
1 with open("data.txt", "r", encoding="utf-8") as f:
2     content = f.read()
3     print(content)
4 # with 块结束后自动关闭文件，不需要写 f.close()
```

4. 写入文件

4.1 write() 写入字符串

```
python
```

```
1 with open("output.txt", "w", encoding="utf-8") as f:
2     f.write("Hello, World!\n")
3     f.write("第二行内容")
```

④ Note

'w' 模式会覆盖原文件。如果不想覆盖，使用 'a' 追加模式。

4.2 写入多行

```
1 lines = ["第一行\n", "第二行\n", "第三行\n"]
2 with open("output.txt", "w", encoding="utf-8") as f:
3     f.writelines(lines)  # 写入列表中的多个字符串
```

5. 常见文件操作示例

5.1 复制文件

```
1 with open("source.txt", "r", encoding="utf-8") as src:
2     content = src.read()
3 with open("dest.txt", "w", encoding="utf-8") as dst:
4     dst.write(content)
```

5.2 统计文件行数

```
1 with open("data.txt", "r", encoding="utf-8") as f:
2     lines = f.readlines()
3 print("总行数:", len(lines))
```

5.3 逐行处理大文件（节省内存）

对于很大的文件，不要用 `read()` 或 `readlines()` 一次加载全部内容，而是逐行迭代：

```
python
1 with open("big_file.txt", "r", encoding="utf-8") as f:
2     for line in f:
3         # 处理每一行
4         print(line.strip())
```

6. 课堂练习（文件操作）

1. **写入文件**：让用户输入多行内容，直到输入空行结束。将用户输入的所有内容写入到 `user_input.txt` 中，每行前面加上行号（如 `1: 用户输入的内容`）。
2. **读取并统计**：读取一个文本文件 `article.txt`，统计文件中出现了多少个汉字（或英文字母）、多少个数字、多少个空格。假设该文件已存在。
3. **文件合并**：现有两个文件 `a.txt` 和 `b.txt`，将这两个文件的内容合并到 `merged.txt` 中，先写 `a` 的内容，再写 `b` 的内容。

第十二章：异常处理

1. 什么是异常？

定义：异常（Exception）是程序运行时发生的错误。如果没有处理异常，程序会崩溃并显示错误信息（Traceback）。

常见异常类型：

异常名称	触发场景	示例
<code>ZeroDivisionError</code>	除以零	<code>10 / 0</code>
<code>ValueError</code>	值类型正确但内容不合适	<code>int("abc")</code>
<code>TypeError</code>	类型错误，如数字+字符串	<code>5 + "a"</code>
<code>NameError</code>	使用未定义的变量	<code>print(x)</code> 而 <code>x</code> 未定义
<code>IndexError</code>	列表/元组索引超出范围	<code>[1,2][5]</code>
<code>KeyError</code>	字典中不存在的键	<code>{"a":1}["b"]</code>
<code>FileNotFoundError</code>	打开不存在的文件	<code>open("no.txt")</code>

生活类比：异常就像开车时遇到路障。如果没有处理（没有绕行），车就会撞上去（程序崩溃）。如果看到“前方有路障”并提前处理（比如绕道），就能继续安全行驶。

2. `try-except` 捕获异常

定义： `try-except` 结构让程序在出错时不会崩溃，而是执行我们指定的“补救”代码。

语法：

```
1  try:
2      # 可能会出错的代码
3      ...
4  except 异常类型:
5      # 出错时执行的代码
6      ...
```

示例：处理除零错误和输入非数字错误。

```
1  try:
2      num = int(input("请输入一个整数: "))
3      result = 10 / num
4      print(f"10 / {num} = {result}")
5  except ValueError:
```

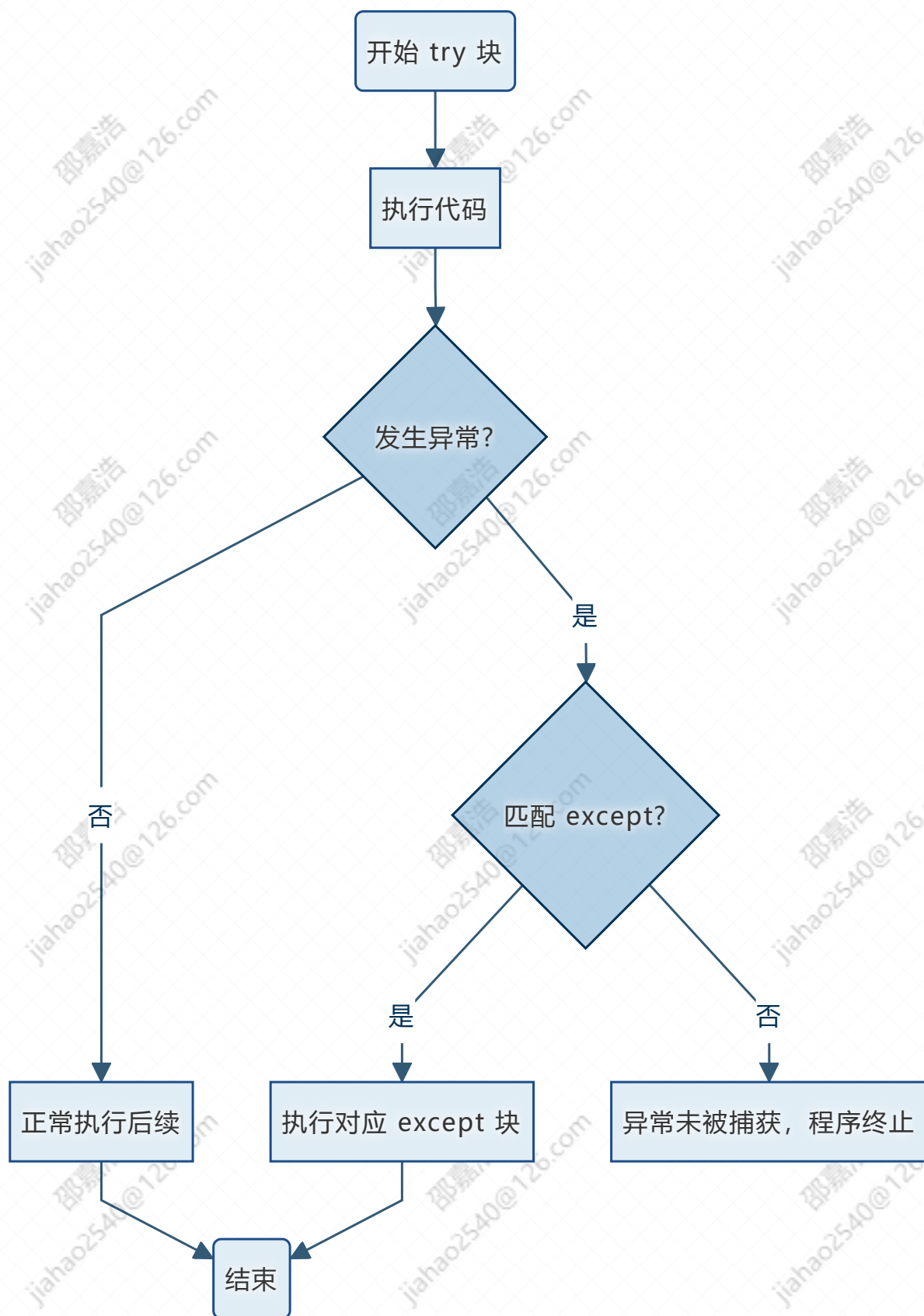


```
6     print("错误：你输入的不是整数！")
7     except ZeroDivisionError:
8         print("错误：除数不能为零！")
```

运行情况：

- 输入 0 → 触发 `ZeroDivisionError` → 输出 错误：除数不能为零！
- 输入 abc → 触发 `ValueError` → 输出 错误：你输入的不是整数！
- 输入 2 → 正常输出 `10 / 2 = 5.0`

流程图：



3. 捕获所有异常（不推荐，但可用）

使用 `except Exception as e` 可以捕获所有类型的异常，并将异常信息存储在变量 `e` 中。

```
python
1 try:
2     x = int("abc")
3 except Exception as e:
4     print(f"出错了: {e}") # 出错了: invalid literal for
    int() with base 10: 'abc'
```

4. `else` 和 `finally`

4.1 `else` —— 没有异常时执行

```
python
1 try:
2     num = int(input("数字: "))
3 except ValueError:
4     print("无效数字")
5 else:
6     print(f"你输入的是 {num}") # 没有异常时才执行
```

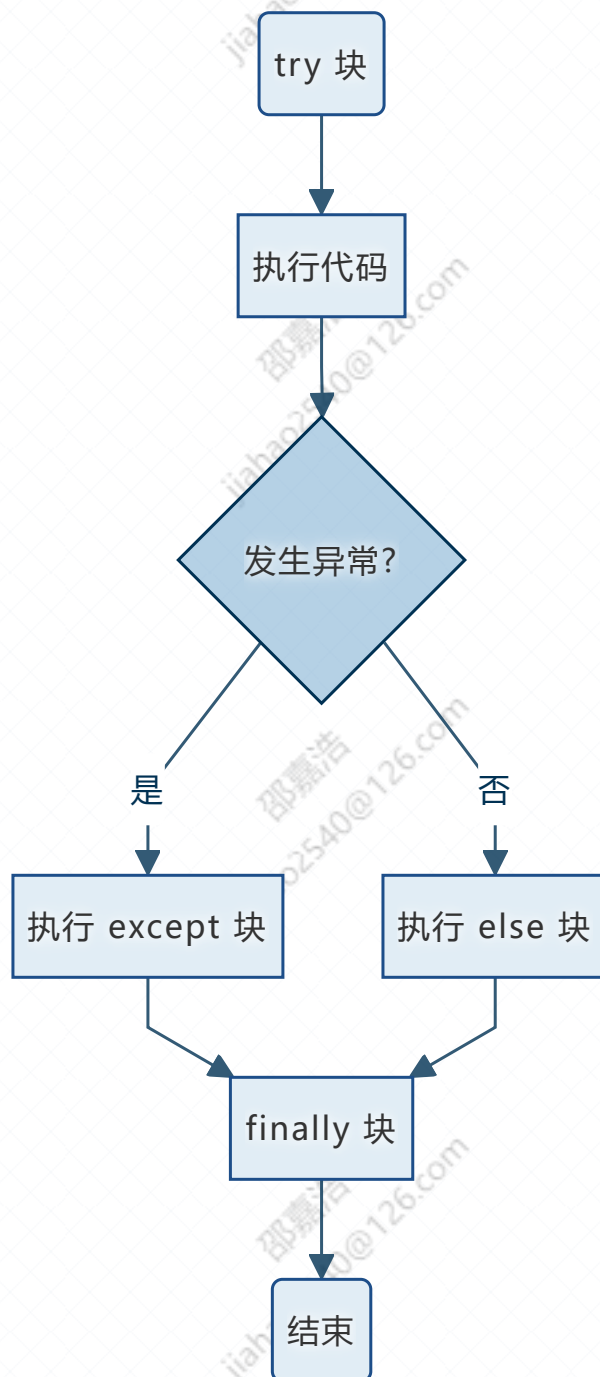
4.2 `finally` —— 无论是否异常都执行

`finally` 常用于释放资源（如关闭文件、断开网络连接）。

```
python
1 try:
2     f = open("data.txt", "r")
3     content = f.read()
4 except FileNotFoundError:
5     print("文件不存在")
```

```
6 finally:
7     print("无论如何都会执行，比如关闭文件")
8     # 如果文件打开了，这里可以关闭 f，但需要先判断 f 是否定义
```

流程图 (try-except-else-finally):



💡 Tip

`else` 和 `finally` 都是可选的。`finally` 常用在必须执行的清理动作，比如关闭文件。

5. 主动抛出异常: `raise`

定义: 当函数接收到不合理的参数时, 可以主动抛出异常, 让调用者处理。

语法: `raise` 异常类型("错误信息")

```
python
1  def set_age(age):
2      if age < 0 or age > 150:
3          raise ValueError("年龄必须在 0~150 之间")
4      print(f"年龄设置为 {age}")
5
6  try:
7      set_age(200)
8  except ValueError as e:
9      print(e)    # 年龄必须在 0~150 之间
```

为什么要主动抛出异常?

- 让函数的调用者知道发生了错误, 由调用者决定如何处理 (捕获还是终止)。
- 比起悄悄返回一个错误值 (如 `-1`), 异常更明确、更安全。

6. 文件操作中的异常处理

文件操作是异常处理的高发区, 常见的异常有: `FileNotFoundError`、`PermissionError`、`UnicodeDecodeError` 等。

```
python
1  filename = input("请输入文件名: ")
2  try:
3      with open(filename, "r", encoding="utf-8") as f:
4          content = f.read()
5          print(content)
6  except FileNotFoundError:
7      print("文件不存在, 请检查文件名")
8  except PermissionError:
```



```
9         print("没有权限读取该文件")
10     except UnicodeDecodeError:
11         print("文件编码不是 UTF-8，无法正确读取")
12     except Exception as e:
13         print(f"其他错误: {e}")
```

7. 课堂练习（异常处理）

1. **安全整数输入**：编写函数 `get_int(prompt)`，使用 `input()` 提示用户输入，如果用户输入的不是整数，则提示“请输入整数”并重新输入，直到用户输入一个合法的整数，然后返回该整数。要求使用 `try-except` 捕获 `ValueError`。
2. **安全的文件读取器**：让用户输入文件名，尝试打开并读取该文件。捕获 `FileNotFoundError` 提示“文件不存在”，捕获 `PermissionError` 提示“无法读取”，其他异常统一提示“未知错误”。最后无论是否成功，都打印“文件操作尝试结束”。
3. **除法函数**：编写函数 `safe_divide(a, b)`，返回 `a/b`。如果 `b==0`，抛出 `ZeroDivisionError` 并自定义错误信息“除数不能为零”；如果参数类型不是数字，抛出 `TypeError`。在主程序中测试，并捕获异常。

综合项目：个人日记本（模块 + 文件操作 + 异常处理）

项目需求：

- 用户可以选择：1. 写日记（追加到文件） 2. 查看所有日记 3. 退出。
- 日记保存到 `diary.txt` 文件中，每一条日记前面加上时间戳。
- 使用模块化设计：将文件操作、时间获取、用户界面分离到不同模块。
- 添加异常处理：文件读写时可能出现的错误（如权限不足、磁盘满等）要捕获并提示。

目录结构：

```
1 diary_project/
2     main.py           # 主程序，用户交互
3     diary_io.py       # 文件读写模块
4     utils.py          # 辅助函数（如获取当前时间字符串）
```

总结与复习

概念	定义（一句话）	要点
模块	一个 <code>.py</code> 文件，用于封装代码。	使用 <code>import</code> 或 <code>from...import</code> 导入；通过 <code>模块名.成员</code> 或直接访问。
<code>__name__</code>	当前模块的名字。直接运行时为 <code>"__main__"</code> ，被导入时为模块名。	常用于 <code>if __name__ == "__main__":</code> 写测试代码。
包	包含多个模块的目录，通常有 <code>__init__.py</code> 。	用于组织大型项目。
内置模块	Python 自带的模块，如 <code>random</code> 、 <code>math</code> 、 <code>datetime</code> 、 <code>os</code> 、 <code>sys</code> 。	直接 <code>import</code> 后使用。
文件操作	使用 <code>open()</code> 打开文件，读/写后需关闭。推荐 <code>with</code> 语句自动关闭。	模式： <code>'r'</code> 读， <code>'w'</code> 写（覆盖）， <code>'a'</code> 追加。
异常	程序运行时发生的错误。	不处理会导致程序崩溃。
<code>try-except</code>	捕获异常，让程序在出错时不崩溃。	可以捕获多个异常类型。
<code>else</code>	没有异常时执行的代码块。	放在 <code>except</code> 之后。
<code>finally</code>	无论是否异常都会执行的代码块，常用于释放资源。	放在 <code>else</code> 之后。
<code>raise</code>	主动抛出异常，用于函数内部检测到不合理输入时。	配合 <code>try-except</code> 使用。

课后作业

1. 模块化统计

创建模块 `stats.py`，实现三个函数：`mean(data)` 求平均值，`median(data)` 求中位数，`mode(data)` 求众数。然后在另一个文件 `test_stats.py` 中导入并测试。

2. 文件复制工具

编写程序，让用户输入源文件名和目标文件名，尝试将源文件内容复制到目标文件。使用异常处理以下情况：

- 源文件不存在 → 提示“源文件不存在”。
- 目标文件无法写入 → 提示“无法写入目标文件”。
- 其他错误 → 打印错误信息。

3. 日记本增强

在综合项目的基础上增加功能：

- 删除所有日记（清空文件）。
- 按日期搜索日记（用户输入日期，如 `2025-01-15`，输出当天写的所有日记）。

4. 猜数字游戏（加异常处理）

使用 `random` 模块生成 1~100 随机数，让用户猜。要求：

- 用户输入的不是整数时，捕获异常并提示“请输入整数”。
- 用户猜中后，询问是否继续玩。
- 记录用户猜测次数和每次猜测的数字（存入文件 `guess_history.txt`）。